SYSTEM V是一些unix系统特性的标准，也就是早期就常出现的一些通信标准

提供了成套的IPC通信接口，包含

> 消息队列 message queue
>
> 共享内存share memory
>
> 信号量semaphore

system V设计理念：在所有进程都能访问的内核空间中，开辟并维护一个相关的结构体空间，用来服务应用程序之间的数据的传递

只不过对于它的访问不需要借助文件IO->系统调用，还是需要时间!!!

主张是让上层应用程序 直接 访问 具备内核空间权限的东西，要怎么实现>

=>应用程序是没有权限访问内核空间的

=>许可证key

1.IPC_key

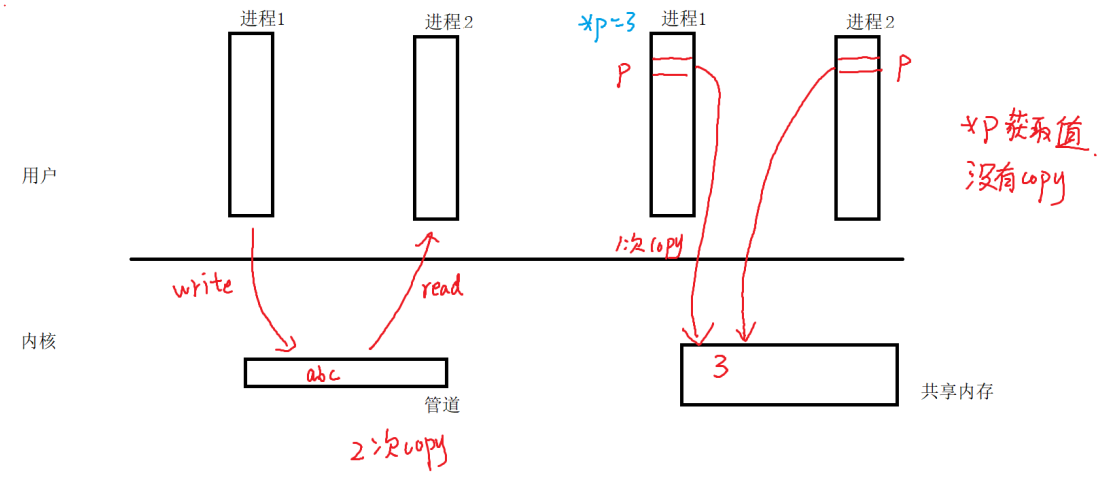
进程间通信许可证是由内核向应用程序颁布的，用于证明应用程序之间拥有借助内核空间进行数据传输资格的一种许可证

```
#include <sys/types.h>
#include <sys/ipc.h>

key_t ftok(const char *pathname, int proj_id);
@pathname:一个文件系统的路径名(必须要是存在的而且有读的权限，在linux文件夹下)
    "/home/china"
@proj_id:整数，这个参数存在的意义是让一个路径也能生成多个key
    ftok只取proj_id的低八位
    eg:
        1
返回值:
    成功，返回一个唯一的system V ipc的key
    失败，-1
eg:
    key_t key = ftok("/home/china",1);
    if(key == -1)
    {

    }
```

2.共享内存



a.ftok生成许可证

b.shmget创建或打开system V共享内存

```
#include <sys/ipc.h>
#include <sys/shm.h>

int shmget(key_t key, size_t size, int shmflg);
@key：由ftok获取
@size：以字节为单位指定共享内存的大小(如果是打开，可以给0)，一般给1024的整数倍
@shmflg：标志位，指定创建或打开共享内存的方式
    如果是创建共享内存:IPC_CREAT |  权限位(0666)
        eg:
            IPC_CREAT | 0777
        IPC_EXCL:测试标志，通常和IPC_CREAT一起用
            IPC_CREAT | IPC_EXCL 如果key对应的共享内存不存在，则创建
                                 如果key对应的共享内存存在，则失败
返回值：
    成功，返回共享内存的id,这个id就唯一表示这个共享内存
    失败,-1
eg:
    int shm_id = shmget(key,4096,IPC_CREAT | IPC_EXCL | 0777);
    if(shm_id == -1)
    {
        //如果是因为共享内存存在而失败，则直接打开
        if(errno == EEXIST)
        {
            shm_id = shmget(key,0,0);
        }
        else
        {
            perror("shmget failed");
            return -1;
        }
    }
```

c.shmat映射到通信进程的各自的进程地址空间中去

```
#include <sys/types.h>
#include <sys/shm.h>

void *shmat(int shmid, const void *shmaddr, int shmflg);
@shmid:要映射的共享内存的id(shmget)
```

```
@shmaddr:指向要银蛇到进程的哪个地址上去
    一般填NULL，有系统分配
@shmflg:
    SHM_RDONLY 只读
    0 可读可写
返回值:
    成功 返回映射后的地址
    失败 (void *) -1
eg:
    int *p = shmat(shm_id,NULL,0);
    if((void*)p == (void*)-1)
    {

    }
```

d.通信

```
int *p = shmat(shm_id,NULL,0);
*p = 3;//(一个程序写入)
printf("%d\n",*p);//(另外一个程序读取)

char *p = shmat(shm_id,NULL,0);
strcpy(p,"nihao");//(一个程序写入)
printf("%s\n",p);//(另外一个程序读取)
```

e.shmdt解映射

```
#include <sys/types.h>
#include <sys/shm.h>

int shmdt(const void *shmaddr);
@shmaddr:需要接触映射的地址

eg:
    shmdt(p);
```

f.shmctl删除共享内存

```
#include <sys/ipc.h>
#include <sys/shm.h>

int shmctl(int shmid, int cmd, struct shmid_ds *buf);
@shmid: 要删除的共享内存的id(shmget)
@cmd:控制命令，不同的命令对应的第三个参数的值会不一行
    IPC_RMID
@buf 保存 第二个参数相关的数据 struct shmid_ds *
    如果cmd == IPC_RMID buf填NULL
返回值
    成功 0
    失败 -1
eg:
    //删除共享内存
    shmctl(shm_id,IPC_RMID,NULL)
```

练习：使用共享内存，在01-shm_write.c中写入你好，在02-shm_read.c中读取

```
01-shm_write.c
    //ftok获取许可证
    //shmget打开或创建共享内存
    //shmat映射 建立你的指针和共享内存的关系
    //写数据
    //shmdt断开映射关系
02-shm_read.c
    //ftok获取许可证
    //shmget打开或创建共享内存
    //shmat映射 建立你的指针和共享内存的关系
    //读取数据
    //shmdt断开映射关系
    //shmctl删除共享内存
```

write

```c
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <errno.h>
#include <stdio.h>
#include <string.h>
int main()
{
    //ftok获取许可证
    key_t key = ftok("/home/china",1);
    if(key == -1)
    {
        perror("ftok failed");
        return -1;
    }
    //shmget打开或创建共享内存
    int shm_id = shmget(key,4096,IPC_CREAT | IPC_EXCL | 0777);
    if(shm_id == -1)
    {
        //如果是因为共享内存存在而失败，则直接打开
        if(errno == EEXIST)
        {
            shm_id = shmget(key,0,0);
        }
        else
        {
            perror("shmget failed");
            return -1;
        }
    }
    //shmat映射 建立你的指针和共享内存的关系
    char *p = shmat(shm_id,NULL,0);
    if((void*)p == (void*)-1)//失败
    {
        perror("shmat failed");
        return -1;
    }
    //写数据
    strcpy(p,"nihao");
    //shmdt断开映射关系
```

```
    shmdt(p);
}
```

read

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <errno.h>
#include <stdio.h>
#include <string.h>
int main()
{
    //ftok获取许可证
    key_t key = ftok("/home/china",1);
    if(key == -1)
    {
        perror("ftok failed");
        return -1;
    }
    //shmget打开或创建共享内存
    int shm_id = shmget(key,4096,IPC_CREAT | IPC_EXCL | 0777);
    if(shm_id == -1)
    {
        //如果是因为共享内存存在而失败，则直接打开
        if(errno == EEXIST)
        {
            shm_id = shmget(key,0,0);
        }
        else
        {
            perror("shmget failed");
            return -1;
        }
    }
    //shmat映射 建立你的指针和共享内存的关系
    char *p = shmat(shm_id,NULL,0);
    if((void*)p == (void*)-1)//失败
    {
        perror("shmat failed");
        return -1;
    }
    //读取数据
    printf("%s\n",p);
    //shmdt断开映射关系
    shmdt(p);
    //删除共享内存
    shmctl(shm_id,IPC_RMID,NULL);
}
```

练习：在内核中创建一个共享内存，把这块共享内存的前4个字节当成是一个int对待，利用父子进程实现并发，具体如下

```
int *p = shmat(...);
*p = 0;
fork();
```

```
父
    while()
    {
        //执行10万次
        (*p)++;
        /*
            存在刚好两个进程同时读取数据的情况
            汇编指令 i++分成三条指令
            1.取i的值放到寄存器
            2.将寄存器的值+1
            3.将寄存器的值存回到i
            =>不是原子操作
            原子操作(不可以被打断)
        */
    }
    //wait子进程结束
    printf("%d\n",*p);
子
    while()
    {
        //执行10万次
        (*p)++;
    }
```

3.信号量

在并发执行多个进程的时候，由于系统资源存在公共资源，并且由些公共资源对访问的数量有限制，因为当多个进程同时访问一个 有限制的共享资源的时候，避免不了竞争

信号量本质：数字，在访问受限制的共享资源之前，必须先访问并获得信号量，如果无法获取信号量，只能进入等待或放弃对共享资源的访问

信号量->厕所门的锁

一个进程或一个线程可以在某个信号量上执行三个操作

- 创建信号量，要求创建者立马指定信号量的初值
- 等待一个信号量(lock) p上锁

```
该操作需要测试这个信号量是否被别人获取了
    如果信号量已经被别人获取，则进入等待
    如果信号量没有被别人获取，则获取信号量 =>
        你就可以访问这个资源
        访问资源的代码


p
操作资源(*p)++    ->临界区
v
```

- 释放一个信号量(unlock) v解锁

```
当你获取信号量之后，并访问完共享资源后，需要立即释放信号量
=>因为别人可能在等待信号量
```

p操作作为可访问资源减操作，v操作作为可访问资源增加操作

3.1 信号量相关的API函数

a.ftok：获取system V ipc对象的key

b.semget:用来创建或打开一个system V信号量集

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

int semget(key_t key, int nsems, int semflg);
@key:ftok获取
@nsems：你要创建的信号量集合中信号量的个数
@semflg：标志位
    创建：IPC_CREAT|权限位(0777)
        IPC_EXCL
        有两种情况
        1.成功
            信号量集合不存在，创建一个信号量集合
        2.失败
            失败的原因errno == EEXIST
            已经创建过了
            之后打开信号量集合
    打开 0
返回值：
    成功 返回一个信号量集合的id
    失败 -1
eg:
    int sem_id = semget(key,5,IPC_CREAT | IPC_EXCL | 0777);
    if(sem_id == -1)
    {
        //如果是因为信号量集合存在而失败，则直接打开
        if(errno == EEXIST)
        {
            sem_id = semget(key,5,0);//如果是打开会自动忽略第二个参数
        }
        else
        {
            perror("semget failed");
            return -1;
        }
    }
```

c.semctl：控制操作(设置或获取信号量集中某个或某些信号量的值)

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

int semctl(int semid, int semnum, int cmd, ...);
@semid:要操作的信号量集合的id(semget)
@semnum:要操作的信号量集中的哪个信号量的下标[0,nsems-1]
@cmd:命令
    IPC_STAT    获取属性
    IPC_SET     设置属性
    IPC_RMID    删除信号量集
    GETALL      获取信号量集中所有信号量的值
```

```
    SETALL      设置信号量集中所有信号量的值
    GETVAL      获取下标semnum的信号量的值
    SETVAL      设置下标semnum的信号量的值
    ....
@...:根据不同的命令，第四个参数也会不一样
    union semun {
        int              val;    /* Value for SETVAL */
        struct semid_ds *buf;    /* Buffer for IPC_STAT, IPC_SET */
        unsigned short  *array;  /* Array for GETALL, SETALL */
        struct seminfo  *__buf;  /* Buffer for IPC_INFO
        (Linux-specific) */
    };
    cmd == IPC_RMID 不需要第四个参数
        semctl(sem_id,0,IPC_RMID);
    cmd == SETVAL
        int val = 1;
        semctl(sem_id,0,SETVAL,val);
        //设置sem_id所表示的信号量集合中下标为0的信号量的值为val
    cmd == GETVAL
        int val = semctl(sem_id,0,GETVAL);
        //val就表示semid所表示的信号量集合中下标为0的哪个信号量的值
    cmd == SETALL
        unsigned short array[5] = {1,0, 0 ,0 ,0};
        int val = semctl(sem_id,0,SETALL,array);
    cmd == GETALL
        unsigned short array[5];//准备空间，用来保存信号量集的值
        int val = semctl(sem_id,0,GETALL,array);
返回值：
    根据命令的不同，返回的结果的含义也不一样
    0   成功
    -1  失败
```

练习：创建一个system v信号量集合(5个信号量)，给所有的信号量设置初值，并且获取第三个信号量的值，并打印

d.semop:PV操作

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>

int semop(int semid, struct sembuf *sops, size_t nsops);
@semid:semget的返回值
@sops：指定信号量操作信息的结构体数组
    struct sembuf
    {
        unsigned short sem_num;  /* semaphore number */
            //要进行操作的信号量在信号量集合中的下标
        short          sem_op;   /* semaphore operation */
            /*
                >0  v操作 unlock
                ==0 看是否阻塞 自己试一下
                <0  p操作 lock
            */
        short          sem_flg;  /* operation flags */
            /*
                0   表示默认，如果p操作失败,就会阻塞
```

```
                IPC_NOWAIT  非阻塞等待，不等待
                SEM_UNDO    撤销，为了防止进程带锁退出
                            用于父子进程(没有血缘关系别用)
            */
    };
@nsops：指定第二个参数sops数组中元素的个数
返回值：成功0 失败-1

int semtimedop(int semid, struct sembuf *sops, size_t nsops,
const struct timespec *timeout);
```

eg:

```
/*
    p
    sem_id:semget获取的
    index:要操作的信号量集中的哪个信号量的下标
*/
void sem_p(int sem_id,int index)
{
    struct sembuf buf;
    buf.sem_num = index;//要操作的信号量的下标
    buf.sem_op = -1;//资源数-1
    buf.sem_flag = 0;//阻塞等待，直到拿到信号量
    semop(sem_id, &buf, 1);
}

/*
    v
    sem_id:semget获取的
    index:要操作的信号量集中的哪个信号量的下标
*/
void sem_v(int sem_id,int index)
{
    struct sembuf buf;
    buf.sem_num = index;//要操作的信号量的下标
    buf.sem_op = +1;//资源数+1
    buf.sem_flag = 0;//阻塞等待，直到拿到信号量
    semop(sem_id, &buf, 1);
}

使用
    //p 上锁
    sem_p(sem_id,0);
    //临界区->操作共享资源(*p)++
    //v 解锁
    sem_v(sem_id,0);
```

练习：把共享内存的最后一个练习，加锁，让最后的结果为20W

查看IPC

```
ipcs        #查看所有IPC对象
ipcs -a     #查看所有IPC对象
ipcs -m     #查看共享内存对象
ipcs -s     #查看信号量对象
```

删除IPC对象

```
ipcrm -M key    #根据指定键值key,删除指定的共享内存
ipcrm -m id     #根据id,删除指定的共享内存

ipcrm -S key    #根据指定键值key,删除指定的信号量
ipcrm -s id     #根据id,删除指定的信号量
```

思考题：

1.在遇到共享内存在不同的进程或线程中访问的时候，考虑避免竞争，用什么方式访问

2.现在有5个共享资源ABCDE，需要被保护，决定用一个信号量来保存5个共享资源，这样设计有什么问题

3.现在有5个共享资源ABCDE，需要被保护，决定用五个信号量来保存5个共享资源，这样设计有什么问题

4.若系统有一个资源，资源数为7，有多个进程均需要使用2个资源，规定每个进程一次仅允许使用1个资源，则最多允许多个进程参与竞争，而不会发生死锁